

DEV_IMPL_GUIDE - FOUNDATION_AUTH

1. Purpose

This guide explains how to implement the SOPHIX authentication and authorization foundation.

Use it with FOUNDATION_AUTH.md.

This is a provisioning/runtime-setup guide. It is intentionally written as infrastructure setup steps rather than a business request execution flow.

2. What You Are Building

You are deploying and configuring the identity foundation used by all SOPHIX modules:

- Keycloak realm.
- Operators/tenants.
- Users and service accounts.
- Roles and groups.
- JWT claims expected by applications.
- Internal service authentication.
- High-availability runtime with two Keycloak nodes.

3. Step 1 - Prepare Infrastructure

Provision:

Component	Minimum medium setup
Keycloak app nodes	2 nodes or pods
PostgreSQL	Shared foundation DB cluster
Load balancer	1 internal HTTPS endpoint
TLS certificates	Internal trusted certificate
Secrets store	Vault, Kubernetes secrets, AWS Secrets Manager, or equivalent

Do not use local H2 or file-based storage.

4. Step 2 - Install Keycloak

Recommended process:

1. Install the approved Keycloak version.
2. Configure both nodes to use the same PostgreSQL database.
3. Configure hostname and HTTPS.
4. Put the nodes behind the load balancer.
5. Enable health checks.
6. Enable metrics.
7. Confirm both nodes can serve login/token requests.

Verification:

```
curl -k https://auth.internal.sophix/health/ready
```

Expected:

```
{
  "status": "UP"
}
```

5. Step 3 - Create Realm

Create the SOPHIX realm:

```
sophix
```

Configure:

- Access token lifetime.
- Refresh token lifetime.
- Password policy.
- Login brute-force protection.
- Required actions for password reset if used.
- Admin event logging.

6. Step 4 - Create Clients

Create clients for:

Client	Type	Purpose
sophix-bo-ui	Public/confidential depending on UI architecture	Back-office UI login.
sophix-api-gateway	Confidential	API gateway token validation/client credentials.
subscription-workflow-service	Confidential service account	SUB-WF service calls.
subscription-management-service	Confidential service account	SUB-LM internal calls.
billing-service	Confidential service account	Billing internal calls.
fulfillment-service	Confidential service account	Fulfillment internal calls.

For service clients:

- Enable service accounts.
- Disable browser login.
- Store client secret in secrets manager.

7. Step 5 - Create Roles And Groups

Create the roles/groups defined in the DD, including:

```
SUBSCRIPTION_ADMIN  
SUBSCRIPTION_OPERATOR  
CALL_CENTER  
CUSTOMER_SELF_SERVICE  
BILLING_INTERNAL  
SCHEDULER_INTERNAL  
FINANCE_WRITE_OFF_APPROVER  
CHURN_SAVE_AGENT  
SUBSCRIPTION_OPS_RECOVERY  
COOLING_OFF_SUPERVISORS  
SALES_AGENT
```

Map operator-specific groups:

```
CALL_CENTER -> wik-call-center-tier1  
COOLING_OFF_SUPERVISORS -> wik-cooling-off-supervisors
```

Keep logical SOPHIX role names stable. Operator-specific group names can vary.

8. Step 6 - Configure Token Claims

Applications need claims such as:

```
{  
  "sub": "user_123",  
  "preferred_username": "sales-agent-wik-042",  
  "operator_code": "WIK",  
  "sophix_roles": ["SALES_AGENT"],  
  "sophix_groups": ["wik-sales-agents"]  
}
```

```
}
```

Implementation rule:

- APIs must authorize from token claims or introspection.
- APIs must not trust actor role sent in request body.

9. Step 7 - Application Integration

Each SOPHIX service must:

1. Validate JWT signature.
2. Validate issuer.
3. Validate audience/client.
4. Extract operator scope.
5. Extract SOPHIX roles/groups.
6. Reject unauthorized calls with 403.

Sample middleware checks:

```
required role: SALES_AGENT
operator scope: WIK
token roles contains SALES_AGENT
token operator_code == WIK
```

10. Step 8 - Service-To-Service Auth

For internal calls:

1. Use client credentials flow.
2. Cache service token until near expiry.
3. Send `Authorization: Bearer <token>`.
4. Downstream service validates token and required service role.

Do not use shared static API keys between modules unless explicitly approved as a temporary migration bridge.

11. Step 9 - Tests

Test:

- Login returns valid JWT.
- Invalid token is rejected.
- Expired token is rejected.
- User without role gets 403.
- Service client can call allowed internal endpoint.
- Service client cannot call unrelated module endpoint.
- Operator WIK user cannot access YASSN data.
- One Keycloak node down does not break token issuance.

12. Common Mistakes

- Trusting `initiatingActorRole` from request body.
- Creating operator-specific role names in application code.
- Running Keycloak with local file/H2 storage.
- Forgetting sticky/session behavior where required.
- Not backing up Keycloak DB.

- Sharing one client secret across all services.